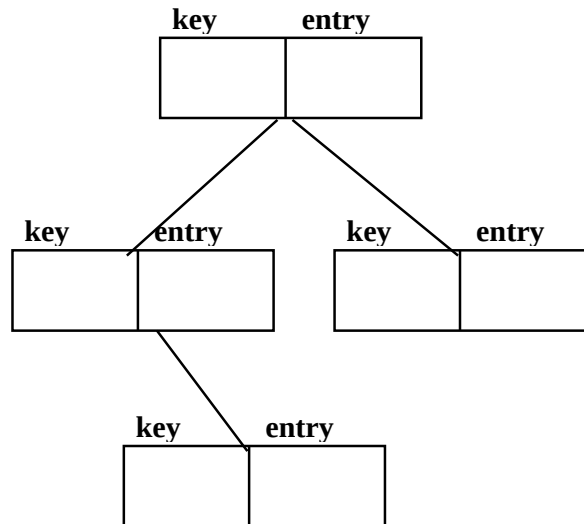


directory or employee directory. In the key, we have the name of the person and the entry contains the address, telephone number and the remaining information. In our AVL tree, we will store this data in the nodes. Though, the search will be on the key, yet as we already noticed that the insert is proportional to  $\log n$ . Being a balanced tree, it will not become degenerated balance tree. The objective of AVL tree is to make the binary trees balanced. Therefore the find will be  $\log n$ . Similarly the time required for the removal of the node is proportional to  $\log n$ .



We have discussed all the five implementations. In some implementations, time required is proportional to some constant time. In case of a sorted list, we have to search before the insertion. However for an unsorted list, a programmer will insert the item in the start. Similarly we have seen the data structure where insertions, deletions and searches are proportional to  $n$ . In link list, insertions and deletions are proportional to  $n$  whereas search is  $\log n$ . It seems that  $\log n$  is the lower limit and we cannot reduce this number more.

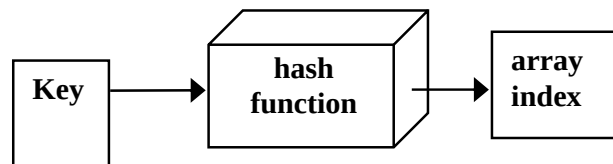
Is it true that we cannot do better than  $\log n$  in case of table? Think about it and send your proposals. So far we have find, remove and insert where time varies between constant and  $\log n$ . It would be nice to have all the three as constant time operations. We do not want to traverse the tree or go into long loops. So it is advisable to find the element in first step. If we want to insert or delete, it should be done in one step. How can we do that? The answer is Hashing.

## Hashing

The hashing is an algorithmic procedure and a methodology. It is not a new data structure. It is a way to use the existing data structure. The methods- *find*, *insert* and *remove* of table will get of constant time. You will see that we will be able to do this in a single step. What is its advantage? If we need table data structure in some program, it can be used easily due to being very efficient. Moreover, its operations are of constant time. In the recent lectures, we were talking about the algorithms and procedures rather than data structure. Now we will discuss about the strategies and methodologies. Hashing is also a part of this.

We will store the data in the array but *TableNodes* are not stored consecutively. We are storing the element's data in the *TableNodes*. You have seen the array implementation of the *Table* data structure. We have also seen how to make the data sorted. There will be no gap in the array positions whether we use the sorted or unsorted data. This means that there is some data at the 1<sup>st</sup> and 2<sup>nd</sup> position of array and then the third element is stored at the 6<sup>th</sup> position and 4<sup>th</sup> and 5<sup>th</sup> positions are empty. We have not done like this before. In case of link list, it is non-consecutive data structure with respect to memory.

In Hashing, we will internally use array. It may be static or dynamic. But we will not store data in consecutive locations. Their place of storage is calculated using the key and a *hash function*. Hash function is a new thing for you. See the diagram below:



We have a key that may be a name, or roll number or login name etc. We will pass this key to a hash function. This is a mathematical function that will return an array index. In other words, an integer number will be returned. This number will be in some range but not in a sequence.

Keys and entries are scattered throughout the array. Suppose we want to insert the data of our employee. The key is the name of the employee. We will pass this key to the hash function which will return an integer. We will use this number as array index. We will insert the data of the employee at that index.

|     | key | entry |
|-----|-----|-------|
|     |     |       |
| 4   |     |       |
|     |     |       |
| 10  |     |       |
|     |     |       |
| 123 |     |       |
|     |     |       |

The insert will calculate place of storage and insert in *TableNode*. When we get a new data item, its key will be generated with the help of hash function to get the array

index. Using this array index, we insert the data in the array. This is certainly a constant time operation. If our hash function is fast, the insert operation will also rapid. It will take only one step to perform this.

Next we have *find* method. It will calculate the place of storage and retrieve the entry. We will get the key and pass it to the hash function and obtain the array index. We get the data element from that array position. If data is not present at that array position, it means data is not found. We do not need to find the data at some other place. In case of binary search tree, we traverse the tree to find the element. Similarly in list structure we continue our search. Therefore find is also a constant time operation with Hashing.

Finally, we have *remove* method. It will calculate the place of storage and set it to null. That means it will pass the key to the hash function and get the array index. Using this array index, it will remove the element.

## Examples of Hashing

Let's see some examples of hashing and hash functions. With the help of these examples you will easily understand the working of *find*, *insert* and *remove* methods.

Suppose we want to store some data. We have a list of some fruits. The names of fruits are in string. The key is the name of the fruit. We will pass it to the hash function to get the hash key.

Suppose our hash function gave us the following values:

|                         |   |   |
|-------------------------|---|---|
| HashCode ("apple")      | = | 5 |
| hashCode ("watermelon") | = | 3 |
| hashCode ("grapes")     | = | 8 |
| hashCode ("cantaloupe") | = | 7 |
| hashCode ("kiwi")       | = | 0 |
| hashCode ("strawberry") | = | 9 |
| hashCode ("mango")      | = | 6 |
| hashCode ("banana")     | = | 2 |

Our hash function name is *hashCode*. We pass it to the string "apple". Resultantly, it returns a number 5. In case of "watermelon" we get the number 3. In case of "grapes" there is number 8 and so on. Neither we are sending the names of the fruits in some order to the function, nor is function returning the numbers in some order. It seems that some random numbers are returned. We have an array to store these strings. Our array will look like as:

|   |            |
|---|------------|
| 0 | kiwi       |
| 1 |            |
| 2 | banana     |
| 3 | watermelon |
| 4 |            |
| 5 | apple      |
| 6 | mango      |
| 7 | cantaloupe |
| 8 | grapes     |
| 9 | strawberry |

We store the data depending on the indices got from the *hashCode*. The array size is 10. In case of apple, we get the index 5 from *hashCode* so “apple” is stored at array index 5. As we are dealing with strings, so the array will be an array of strings. The “watermelon” is at position 3 and so on every element is at its position. This array will be in the private part of our data structure and the user will not know about it. If our array is *table* then it will look like as under:

```
table[5] = "apple"
table[3] = "watermelon"
table[8] = "grapes"
table[7] = "cantaloupe"
table[0] = "kiwi"
table[9] = "strawberry"
table[6] = "mango"
table[2] = "banana"
```

We will store our data in the Table array using the string copy. The user is storing the data using the names of the fruits and wants to retrieve or delete the data using the names of fruits. We have used the array for storage purposes but did not store the data consecutively. We store the data using the hash function which provides us the array index. You can note that there are gaps in the array positions.

Similarly we will retrieve the data using the names of fruit and pass it to the *hashCode* to get the index. Then we will retrieve the data at that position. Consider the table array, it seems that we are using the names of fruits as indices.

```
table["apple"]
table["watermelon"]
table["grapes"]
table["cantaloupe"]
table["kiwi"]
```

```
table["strawberry"]
table["mango"]
table["banana"]
```

We are using the array as table ["apple"], table ["watermelon"] and so on. We are not using the numbers as indices here. Internally we are using the integer indices using the *hashCode*. Here we have used the fruit names as indices of the array. This is known as associative array. Now this is the internal details that we are thinking it as associative array or number array.

Let's discuss about the *hashCode*. How does it work internally? We pass it to strings that may be persons name or name of fruits. How does it generate numbers from these? If the keys are strings, the hash function is some function of the characters in the strings. One possibility is to simply add the ASCII values of the characters. Suppose the mathematical notation of hash function is  $h$ . It adds all the ASCII values of the string characters. The characters in a string are from 0 to  $length - 1$ . Then it will take mod of this result with the size of the table. The size of the table is actually the size of our internal array. This formula can be written mathematically as:

$$h(str) = \left( \sum_{i=0}^{length-1} str[i] \right) \% TableSize$$

*Example :  $h(ABC) = (65 + 66 + 67) \% TableSize$*

Suppose we use the string "ABC" and try to find its hash value. The ASCII values of A, B and C are 65, 66 and 67 respectively. Suppose the tableSize is 55. We will add these three numbers and take mod with 55. The result (3.6) will be the hash value. To represent character data in the computer ASCII codes are used. For each character we have a different bit pattern. To memorize this, we use its base 10 values. All the characters on the keyboard like \$, %, ' have ASCII values. You can find the ASCII table in your book or on the internet.

Let's see the C++ code of *hashCode* function.

```
int hashCode( char* s )
{
    int i, sum;
    sum = 0;
    for(i=0; i < strlen(s); i++ )
        sum = sum + s[i]; // ascii value
    return sum % TABLESIZE;
}
```

The return type of *hashCode* function is an integer and takes a pointer to character. It declares local variable  $i$  and  $sum$ , then initializes the  $sum$  with zero. We use the *strlen* function to calculate the length of the string. We run a loop from 0 to  $length - 1$ . In the loop, we start adding the ASCII values of the characters. In C++, characters are

stored as ASCII values. So we directly add  $s[i]$ . Then in the end, we take mod of sum with  $TABLESIZE$ . The variable  $TABLESIZE$  is a constant representing the size of the table.

This is the one of the ways to implement the hash function. This is not the only way of implementing hash function. The hash function is a very important topic. Experts have researched a lot on hash functions. There may be other implementations of hash functions.

Another possibility is to convert the string into some number in some arbitrary base  $b$  ( $b$  also might be a prime number). The formula is as:

$$h(str) = \left( \sum_{i=0}^{length-1} str[i] \times b^i \right) \% T$$

$$Example: h(ABC) = (65b^0 + 66b^1 + 67b^2) \% T$$

We are taking the ASCII value and multiply it by  $b$  to the power of  $i$ . Then we accumulate these numbers. In the end, we take the mod of this summation with  $tableSize$  to get the result. The  $b$  may be some number. For example, we can take  $b$  as a prime number and take 7 or 11 etc. Let's take the value of  $b$  as 7. If we want to get the hash value of ABC using this formula:

$$H(ABC) = (65 * 7^0 + 66 * 7^1 + 67 * 7^2) \bmod 55 = 45$$

We are free to implement the hash function. The only condition is that it accepts a string and returns an integer.

If the keys are integers,  $key \% T$  is generally a good hash function, unless the data has some undesirable features. For example, if  $T = 10$  and all keys end in zeros, then  $key \% T = 0$  for all keys. Suppose we have employee ID i.e. an integer. The employee ID may be in hundreds of thousand. Here the table size is 10. In this case, we will take mod of the employee ID with the table size to get the hash value. Here, the entire employee IDs end in zero. What will be the remainder when we divide this number with 10? It will be 0 for all employees. So this hash function cannot work with this data. In general, to avoid situations like this,  $T$  should be a prime number.

## Data Structures

### Lecture No. 42

#### Reading Material

Data Structures and Algorithm Analysis in C++

Chapter. 5  
5.1, 5.2, 5.4.1, 5.4.2

#### **Summary**

- Collision
- Linear Probing

In the previous lecture, we were discussing about the hash functions. The hash algorithm depends on the hash function. The hash function generates the array index to enable us to insert data in the table array. We have seen two examples of hash functions in the previous lecture. Both the functions use the ASCII values of characters to generate the index. Here the question arises how can we implement the hash function in case of having integer data? We may have employee ID, user ID or student ID as integers. Here we may take mod with some number or table size and the result is used as an array index.

If the keys are integers then  $key \% T$  is generally a good hash function unless the data has some undesirable features. If we want to store the employee record, user record or student record in the table, this can be done through hash function. We take the mod of the value with the  $T$ . The value of  $T$  may be 10, 15 or 100 depending on the requirements. There may be some problem. For example, if  $T = 10$  and all keys end in zeros, then  $key \% T = 0$  for all keys. The hash function gives 0 for all the keys, used as array index. Now it is a problem. We cannot store our values as all the records have same index i.e. 0. In general, to avoid such situations,  $T$  should be a prime number. Internally, we have to store the data in the array and there is complete freedom to use this array by taking the size of our own choice.

We have to store some data in the array. As the array is private, we will decide about its size on our own. We will take the size of the array in prime numbers. To store 100 records, we will take prime number near 100. We will select this prime number as *MAXTABLESIZE*. Then we will use this number in our hash function. This will help resolve the problem arising due to situation where all keys end with 0. Using the prime number, the values from the hash function will not be 0 for all the keys.

With the help of prime number, we cannot solve this problem completely. Similarly, it cannot be made sure that the values from the hash function are unique for all the keys. Sometimes, we may have same index for two different keys. This phenomenon is known as collision i.e. the hash values are same of two different keys. How can we solve this collision problem?

#### **Collision**

Collision takes place when two or more keys (data items) produce the same index.

Let's see the previous example of storing the names of fruits. Suppose our hash function gives us the following values:

|                    |   |   |   |            |
|--------------------|---|---|---|------------|
| hash("apple")      | = | 5 | 0 | kiwi       |
| hash("watermelon") | = | 3 |   |            |
| hash("grapes")     | = | 8 | 1 |            |
| hash("cantaloupe") | = | 7 |   |            |
| hash("kiwi")       | = | 0 | 2 | banana     |
| hash("strawberry") | = | 9 |   |            |
| hash("mango")      | = | 6 | 3 | watermelon |
| hash("banana")     | = | 2 |   |            |
|                    |   |   | 4 |            |
|                    |   |   | 5 | apple      |
|                    |   |   | 6 | mango      |
|                    |   |   | 7 | cantaloupe |
|                    |   |   | 8 | grapes     |
|                    |   |   | 9 | strawberry |

We store these data items in the respective index in the array. In the above example, the index given by the hash function does not collide with any other entry. Suppose we want to add another fruit “honeydew” in it. When “honeydew” is passed to the hash function, we get the value 6 i.e.

$$\text{hash}(\text{"honeydew"}) = 6$$

This is not the responsibility of the hash function to see the data in the array before generating the index. Hash function is generally a mathematical formula that takes the keys and returns a number. It is responsibility of the caller to find its solution. We have already “mango” at position 6. The user of our ADT calls the insert function giving the value “honeydew”. We call the hash function to find out its index that comes out to be 6. Now the problem is this that position 6 is already occupied. What should we do to avoid it?

There are a lot of solutions of this problem. These solutions can be divided into two main categories. One type of solutions is the changing of the hash function. Even with the introduction of a new function, it is not guaranteed that there will be no collision with future data. Second option is that we live with the collision and do something to resolve it.

The definition of collision is:

“When two values hash to the same array location, this is called a *collision*”

We cannot say that the usage of this hash function will not result in collision



especially when the data is changing. Collisions are normally treated as a phenomenon of “first come, first served”, the first value that hashes to the location gets it. We have to find something to do with the second and subsequent values that hash to the same location.

As we have seen above in the example that at position 6, there is the data item *mango* while another item *honeydew* is trying to acquire the same position. First come first served means that the *mango* will remain at its position and we will do something for *honeydew*. We will see three solutions for this problem:

**Solution #1:** Search for an empty location

- Can stop searching when we find the value or an empty location.
- Search must be wrap-around at the end.

Let's apply this solution on the above example. We have *mango* at position 6. Now the position for *honeydew* is also 6. We find another empty location in the array. Keep in mind that in hashing, we do not store data at consecutive positions. Rather, data is scattered. We know that there are some empty locations in the array. We do not want to refuse the user that we cannot store the data due to the occupation of the position by some other item. Therefore, we will store the data item at the empty location. We will see an example in a short while.

**Solution #2:** Use a second hash function

- ...and a third, and a fourth, and a fifth, ...

We have a primary hash function, we pass it to the string *honeydew*. It returns 6. As the position 6 is occupied, so we call another hash function that is implemented in a different way. This hash function will return an integer. If this location is empty, we will store our data here and the problem is solved. But if the array location at the index number returned by this function is also occupied. Then we will call the third hash function. There is a possibility of collision so we will have three, four, five or more hash functions. When there is a collision, we call these hash functions one by one till the time, there is an index with empty location. We will store the data at this empty location.

**Solution #3:** Use the array location as the header of a linked list of values that hash to this location

In this solution, we will not store the data directly in the array. Our array will be an array of pointers to *TableNode*. We will create a list node and store the data in it. The array will have a pointer to the node. If we want to store some data at location 6, we will store a pointer at location 6 that points to the node containing the data. When we have another data at the position 6, we create a list node and attach it with the previous node. There is a possibility of having a link list from each location of the table. At least, it will have one node.

There are some more strategies for the resolution of this problem. But we will study only the above-mentioned three solutions. Now we will see how we can implement the methods of *Table* and *Dictionary ADT* using the hashing function. Firstly, we will see how the *insert*, *find* and *remove* methods will work with the first solution.

## Linear Probing

The first solution is known as open addressing. When there is a collision, we try to find some other place in our array. This approach of handling collisions is called *open addressing*; it is also known as *closed hashing*. The word open is used with addressing and the word closed is used with hashing. Be careful when naming these. More formally, cells at  $h_0(x)$ ,  $h_1(x)$ ,  $h_2(x)$ , ... are tried in succession where

$$h_i(x) = (\text{hash}(x) + f(i)) \bmod \text{TableSize}, \quad \text{with } f(0) = 0.$$

Here *hash* is our hash function. If there is some collision, we add  $f(i)$  value to it before taking its mod with *TableSize*. The function,  $f$ , is the collision resolution strategy.

We use  $f(i) = i$ , i.e.,  $f$  is a linear function of  $i$ . Thus

$$\text{location}(x) = (\text{hash}(x) + i) \bmod \text{TableSize}$$

The function  $f$  can be any function. So our first implementation of  $f$  is that whatever integer ( $i$ ) we gave to it, it returns it back. That is  $f(i)$  is a linear function. We can implement function  $f$  as it returns the square of  $i$ . That will be quadratic function. Similarly, we can have cubic functions. We are free to implement it.

In the above example, we are given  $x$  for insertion. This can be string such as *honeydew*. We call the  $\text{hash}(x)$  that returns a number. We will see that this location is empty or not. In case of collision, we add  $i$  to this number. The value of  $i$  can be 0,1,2,3 etc. We will take the mod of this value with *TableSize* and try to store the data there. The collision resolution strategy is called *linear probing* as it scans the array sequentially (with wrap around) in search of an empty cell. Let's consider an example to understand how it works. Keep in mind that when there is a collision, some other location in the array is found. This is known as linear probing. Here we are trying to probe the array linearly that where is the empty location.

Let's see an example. Here we have names of birds being stored in the array. We have a larger array. We already have some data in the array that is as under:

|     |         |
|-----|---------|
| ... |         |
| 141 |         |
| 142 | robin   |
| 143 | sparrow |
| 144 | hawk    |
| 145 | seagull |
| 146 |         |
| 147 | bluejay |
| 148 | owl     |
| ... |         |

This is a part of array. There is some data before and after these locations. As shown above, the locations 142, 143, 144, 145, 147 and 148 contains data. Suppose we want to add *seagull* to this hash table. With respect to ADT, someone is using this *Table* or *Dictionary* and calls the *insert* method of this *Table*. Now we have to use the *seagull* as key and have to store its associated data. We will call the *hashCode* function to get the array index and pass it the string “seagull” as:

$\text{hashCode}(\text{“seagull”}) = 143$

Now the location 143 is not empty and we have a bird, *sparrow* at this location. That is  $\text{table}[143]$  is not empty. The *seagull* is different than *sparrow* i.e.  $\text{table}[143] \neq \text{seagull}$ . If it already exists and we are not allowing duplicates, we can say that the data already exists. This is not the case here. We will check the next position,  $143+1$  (here  $i$  is 1) i.e. 144. At location 144, we have a bird *hawk*. So this position is also not empty and not equal to *seagull*. Here also we have different data. Now we check the next position  $143+2$  (here  $i$  is 2) i.e. 145. This location is empty so we store our data at this location. We have two collisions here at 143 and 144 positions. We have found the empty position in the array with single jumps. First we add 1, then 2 and so on.

Suppose we want to add some existing data. As collisions are happening here, so when we get the index value from hash function, the data may not be at that position. We have to follow this linear chain. Suppose you want to add *hawk* to this hash table. The table is as under:

|     |         |
|-----|---------|
| ... |         |
| 141 |         |
| 142 | robin   |
| 143 | sparrow |
| 144 | hawk    |
| 145 | seagull |
| 146 |         |
| 147 | bluejay |
| 148 | owl     |
| ... |         |

We know that *hawk* is already in the table. Also *seagull* is added in the table. We will see that that data that we want to insert already exists or not. First of all, we call the hash function that is *hashCode* as:

$\text{hashCode}(\text{“hawk”}) = 143$

We use this value as the table index to insert this data in the table. The *table[143]* is not empty. Then we check the data at position 143. We have stored *sparrow* here so *table[143] != hawk*. At this point, we cannot decide whether the data item *hawk* exists in the table or not. As we are doing linear probing and there are chances that this data may exist at some other location. Therefore we will check it further taking any final decision. Now we check the next position i.e. 144. It is also not empty but the data here is *hawk* that is *table[144] == hawk*. We have two options here. One is that *hawk* is already in the table, so do nothing. The other option is that user might want to modify the data associated with *hawk*.

The size of our internal array is fixed and we store it in the constant *TableSize*. In linear probing, at the time of collisions, we add one to the index and check that location. If it is also not empty, we add 2 and check that position. Suppose we keep on incrementing the array index and reach at the end of the table. We were unable to find the space and reached the last location of the array. Now what does it mean? Is the table completely filled? There are chances that we started from the position 143. Suppose that the table size is 500. We were unable to find any empty location. Can we say that the array locations before 143 are also not available? There is a possibility that we find some empty spaces there. We will not stop at the end of the table and go to the start of this table array. We will start linear probing from the start and if we find some empty spaces, the data will be stored there. This process is called wrap around. There is a chance that in wrap around, we could not find space and come back to the 143 position. The solution of this problem is to get a larger array. We may want to create an array of size 1000 or some prime number larger than 500. Then copy all the data in this new array. This new data may be inserted with the help of linear probing. The other solution is that we may want to keep multiple arrays. In this case, when one array is filled we go to the second array.

Let's see the example of wrap around process. Suppose we want to add *cardinal* to this hash table. We will use the above table to insert this data. Also suppose that 148 is the last position of this table. We call the *hashCode* function as:

`hashCode("cardinal") = 147`

We get the array index as 147. This position is already filled in the table. The data at this position is *bluejay*. So we will add 1 to the index and check the 148 position. This position is also filled. We do not stop here and go back to the start of the array. We cannot increment the array index further. We will treat the table as circular; after 148 comes 0. Now we will check the position 0. If it is occupied, we will see the position 1 then 2 and so on. We have seen the *insert* method using the linear probing. Now we will see the *find* method.

Suppose we want to find *hawk* in this hash table. We call the *hashCode* as:

`hashCode("hawk") = 143`

The *table[143]* is not empty. However, the data at this position is not *hawk*. So with the linear probing procedure, we add 1 to the index and check the 144 position. This position is also not empty. We compare it with *hawk* that is true. We use the same procedure for looking things up in the table as generally done for inserting them.

Let's have a look on the delete procedure. If an item is placed in  $array[hash(key)+4]$ , the item just before it is deleted. How will probe determine that the "hole" does not indicate the item is not in the array? We may have three states for each location as:

- Occupied
- Empty (never used)
- Deleted (previously used)

Using the linear probe, we insert data where we get some empty space in the array. Firstly, we try to insert it at the index position given by the hash function. If it is occupied, we move to the next position and so on. In this way, we have a chain to follow. Now if an element of the chain is deleted, how can we know that it was filled previously. As we have seen in the above example that *seagull* collided twice before getting a space in the array. Despite having a hash value of 143, it was stored at 145. The three locations 143, 144 and 145 have different data values but are a part of a chain. These three names collide with each other with respect to hash function. Suppose we delete the 2<sup>nd</sup> name i.e. *hawk* due to some reason. Now there are two names, which collide. A space has been created due to the deletion of *hawk*. If we try to search *seagull* using the linear probing, there will be a hash value of 143. It is filled but has different data. We move to the next position that is empty. How can linear probing know that do not stop here because there was some data previously at this position, which has not been deleted. And there is some data after this position, which is part of this chain due to the collisions and that data is *seagull*.

To deal with such situations, we keep three states in our array. 1) It is empty (never used) and has no data in it. 2) It is filled with some legal data and is occupied and 3) it had some data which is now deleted and currently it is empty. We will use this state if we have to go to next position using the linear probing.

One problem with linear probing technique is the tendency to form "clusters". A cluster is a group of items not containing any open slots. The bigger a cluster gets, the more likely the new values will hash into the cluster, and make it even bigger. Clusters cause efficiency to degrade. Suppose we want to add another bird in our table. The hash function returns the index as 143. Now we have already three items that collided at 143. The array locations at 143, 144, 145 are already occupied. So we will insert this new data at 146 using the linear probing. The data is getting gathered instead of scattering because linear probing inserts the data in the next position. It seems as the normal use of the array in which we insert data in the array from first position then next position and so on. It may depend on our data or our hash function. This gathering of data is called clustering.

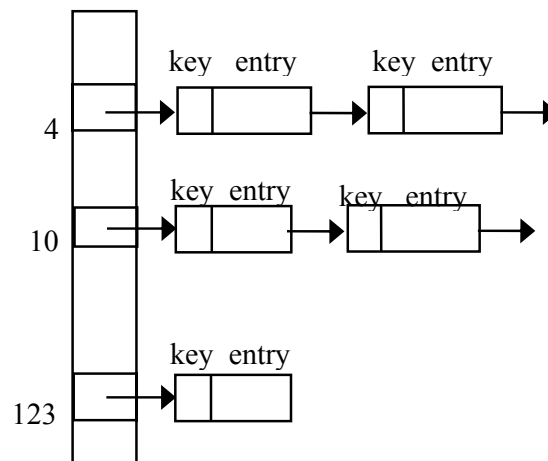
We were trying to store the data in the array in a constant time or in a single step. Similarly the *find* and *remove* methods should be of constant time. That attribute has now been lost. What should we do now? One of the solutions is quadratic probing.

Quadratic probing uses different formula:

- Use  $F(i) = i^2$  (square of  $i$ ) to resolve collisions
- If hash function resolves to  $H$  and a search in cell  $H$  is inconclusive, try  $H + 1^2, H + 2^2, H + 3^2, \dots$

In the quadratic probing when a collision happens we try to find the empty location at  $\text{index} + 1^2$ . If it is filled then we add  $2^2$  and so on. Let's take the above example. We want to insert seagull. The hash function generates the value 143 and this location is already occupied. Now we will start probing. First of all, we add  $1^2$  in the 143 and have 144. It is also not empty. Now we will add  $2^2$  in the 143 and have 147. If it is also occupied, we will add  $3^2$  in 143 and we have 152. The data is now getting scattered. Unfortunately, there are some problems with quadratic probing also.

Let's discuss the third solution that we will use the link list for the collision resolution. Each table position is a linked list. When we are going to insert new data, we will insert the keys and entries anywhere in the list (front easiest). It is shown in the below diagram.



On the left side, we have vertical array that contains the pointers in it. When we insert the first item, we attach a list node. When we have collision, the new item is inserted in the start of the link list. Suppose an item is stored at position 4 and we have another data, requiring the position 4. So there is a collision and we will insert the data at the start of the list.

Let's compare the link list and open addressing.

- Advantages over open addressing:
  - Simpler insertion and removal
  - Array size is not a limitation
- Disadvantage
  - Memory overhead is large if entries are small.

The problem in linear probing is that when our array is full what we should do. This problem can be solved using the link list.

In the next lecture, we will continue our discussion on hashing. We will see an animation. Hashing is very important methodology and can be used in data structures besides *Table* and *Dictionary*.